

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

C02: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks:50

Annexure A

Scenario based

Code of Conduct:

I Dhanya Sri M (192572099) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Question 1 — CPU Scheduling in a Real-Time Ticket Booking System

A railway ticket booking system experiences heavy load during festival seasons. The system handles three categories of processes:

- P1 – Payment gateway transactions (High priority)
- P2 – Seat availability checks (Medium priority)
- P3 – Ticket confirmation and printing (Low priority)
- Burst times vary from 2 ms to 20 ms.

Recently, customers complained that ticket confirmations are taking too long.

Tasks

1. Identify the most suitable scheduling algorithm (FCFS, SJF, Priority, RR) for this scenario.
2. Justify *why* your chosen method improves CPU utilization and reduces customer wait time.
3. Provide a sample execution order for the above categories.
4. Discuss one drawback of this scheduling method in real-time systems.

(10 Marks)

Question 2 — Deadlock in a Banking Transaction Server

A bank server handles the following operations simultaneously:

- T1: Transfers money between accounts
- T2: Updates passbook entries
- T3: Generates monthly statements

Due to simultaneous locking of Account Database (A), Customer Info Database (B), and Transaction Logs (C), the system enters a state where:

- T1 holds A and requests B
- T2 holds B and requests C
- T3 holds C and requests A

Tasks

1. Draw the resource request situation (text-based RAG).
2. Identify whether a deadlock has occurred and justify your answer.

3. Recommend *one specific technique* (avoidance/prevention/recovery) and explain how it fixes the issue.
4. Suggest a redesign for the locking order to avoid future deadlocks.

(10 Marks)

Question 3 — Mobile OS App Switching Lag

A mobile operating system is designed to support smooth app switching.

But users report that:

- Opening WhatsApp delays the background music player.
- Switching between social media apps feels laggy.
- CPU shows high idle times despite multiple running apps.

The OS currently uses Round Robin (quantum = 20 ms) with 20 background apps active.

Tasks

1. Analyze why the CPU remains idle despite many active processes.
2. Determine whether the time quantum is helping or hurting performance.
3. Recommend a better scheduling method or quantum value, with justification.
4. Explain how the change will reduce switching lag and improve system throughput.

(10 Marks)

Question 4 — Deadlock Risk in an Operating System Update

During a Windows OS update, several modules run in parallel:

- M1: Copy update files
- M2: Install drivers
- M3: Update registry
- M4: Validate system integrity

Resources involved:

- R1 = Disk I/O
- R2 = Kernel lock
- R3 = Configuration lock

Recently logs show:

- M1 holds R1, waiting for R2

- M2 holds R2, waiting for R3
- M3 holds R3, waiting for R1

Tasks

1. Identify whether this sequence can lead to a deadlock.
2. If yes, explain the exact circular wait condition.
3. Propose *Banker's algorithm style* resource allocation to avoid risk.
4. Suggest how OS designers can prioritize modules to improve CPU utilization during update.

(10 Marks)

Question 5 — Cloud Server CPU Utilization Analysis

A cloud hosting provider notices:

- 80% of the time, VM processes wait for network or disk
- Only 20% of the time is spent on CPU
- Each server runs 7 VMs

Using the formula:

$$U = 1 - p^n$$

where p = I/O wait probability

Tasks

1. Calculate CPU utilization.
2. Interpret whether the server is underutilized or optimally loaded.
3. Suggest how increasing or decreasing number of VMs affects throughput.
4. Propose two OS-level strategies for improving CPU utilization in this cloud environment.

(10 Marks)

Question 6 — Hospital Emergency System Process Scheduling

A hospital information system manages:

- P1 – Emergency patient registration (Highest Priority)
- P2 – Laboratory report generation (Medium Priority)
- P3 – Pharmacy billing (Low Priority)

During peak hours, pharmacy billing is delayed while emergency requests continue to arrive.

Tasks

1. Identify the scheduling issue occurring in the system.
2. Recommend a suitable CPU scheduling algorithm.
3. Explain how starvation can be prevented.
4. Discuss how your solution improves response time and fairness.

(10 Marks)

Question 7 — E-Commerce Warehouse Synchronization

An e-commerce warehouse has multiple worker threads updating a shared inventory database simultaneously.

Developers observe:

- Incorrect stock counts
- Duplicate order confirmations
- Frequent data inconsistencies

Tasks

1. Identify the synchronization problem.
2. Explain why a critical section is required.
3. Recommend either semaphores or monitors to solve the issue.
4. Explain how the proposed solution maintains data consistency.

(10 Marks)

Question 8 — Video Streaming Service Thread Management

A video streaming platform creates one thread for every client connection.

During a live sports event:

- 60,000 users connect simultaneously.
- CPU utilisation reaches 95%.
- Memory consumption increases rapidly.
- Video buffering becomes frequent.

Tasks

1. Identify the thread-management problem.
2. Recommend a suitable multithreading model.
3. Suggest two optimisations to improve scalability.
4. Explain how your recommendations improve throughput.

(10 Marks)

Question 9 — Smart Traffic Control Deadlock

A smart city traffic management system controls four intersections.

Resources:

- R1 – North Junction
- R2 – East Junction
- R3 – South Junction
- R4 – West Junction

Current situation:

- T1 holds R1 and requests R2.
- T2 holds R2 and requests R3.
- T3 holds R3 and requests R4.
- T4 holds R4 and requests R1.

Tasks

1. Determine whether a deadlock exists.
2. Explain the four necessary conditions for deadlock in this scenario.
3. Recommend one recovery technique.
4. Suggest a redesign that prevents similar deadlocks.

(10 Marks)

Question 10 — University Examination Server Resource Allocation

An online examination server allocates CPU, memory and database connections to student sessions.

At examination time:

- Thousands of students log in simultaneously.
- New resource requests arrive continuously.
- The administrator wants to avoid deadlocks without reducing resource utilisation.

Tasks

1. Explain how Banker's Algorithm can be applied.
2. Describe the information required before granting a request.
3. Explain how the system determines whether it is in a safe state.
4. Discuss one limitation of using Banker's Algorithm in large-scale cloud systems.

(10 Marks)

RUBRICS FOR CALCULATION

Scenario based assessment

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding ; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Marks Evaluation:

Question No.	Understanding of Scenario (20%)	Identification of Key Issues (20%)	Application of Concepts (25%)	Decision Making (20%)	Clarity of Response (15%)	Total Marks(100)
Q1						
Q2						
Q3						
Q4						
Q5						
Q6						

Q7						
Q8						
Q9						
Q10						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Question 1 — CPU Scheduling in a Real-Time Ticket Booking System

1. Most suitable scheduling algorithm: Priority Scheduling

The most suitable algorithm is **Priority Scheduling** because the ticket booking system has different categories of processes with different priorities:

- **P1 – Payment gateway transactions → High Priority**
- **P2 – Seat availability checks → Medium Priority**
- **P3 – Ticket confirmation and printing → Low Priority**

Since payment transactions are more important and time-sensitive, they should be executed before lower-priority tasks.

2. How it improves CPU utilization and reduces customer wait time

- High-priority processes are executed first, so important transactions are completed quickly.
- The CPU does not remain idle when there are ready processes.
- Payment and seat availability operations can be processed quickly during heavy traffic.
- Proper priority assignment helps reduce the waiting time of critical operations.
- Faster processing of important tasks improves the overall responsiveness of the ticket booking system.

3. Sample execution order

- Assume all three processes are ready at the same time:

Execution order:

P1 → P2 → P3

Process	Task	Priority	Execution
P1	Payment gateway	High	1st
P2	Seat availability	Medium	2nd
P3	Ticket confirmation & printing	Low	3rd

Example:

P1 (Payment) → P2 (Seat Check) → P3 (Confirmation & Printing)

This ensures that critical payment operations are handled first.

4. One drawback in real-time systems

The major drawback is **starvation**.

If high-priority processes continuously arrive, the low-priority **P3 (ticket confirmation and printing)** may have to wait for a very long time. This can explain why customers are experiencing delays in ticket confirmation.

Solution: Aging can be used, where the priority of a waiting process is gradually increased to prevent starvation.

Conclusion

Priority Scheduling is the best choice because the processes have clearly defined priorities. It gives faster service to critical operations, improves system responsiveness, and helps manage heavy festival-season loads. However, **starvation of low-priority processes is a major drawback.**

Question 2 — Deadlock in a Banking Transaction Server

1. Resource Allocation Graph (RAG)

Processes: T1, T2, T3

Resources: A = Account DB, B = Customer Info DB, C = Transaction Logs

Text-based RAG:

T1 —holds—> A

T1 —requests—> B

T2 —holds—> B

T2 —requests—> C

T3 —holds—> C

T3 —requests—> A

The cycle is:

$T1 \rightarrow B \rightarrow T2 \rightarrow C \rightarrow T3 \rightarrow A \rightarrow T1$

2. Has deadlock occurred?

Yes, deadlock has occurred.

Each transaction is holding one resource and waiting for another resource held by another transaction.

- T1 holds A and waits for B.
- T2 holds B and waits for C.
- T3 holds C and waits for A.

Therefore, a **circular wait** exists, and none of the transactions can proceed.

3. Recommended technique — Deadlock Prevention

Use **resource ordering**.

Assign an order:

$A \rightarrow B \rightarrow C$

Every transaction must request resources only in this order.

For example, T3 should not hold C and then request A. It must request A before C.

This prevents circular waiting and therefore prevents deadlock.

4. Locking order redesign

Use the same global locking order for all transactions:

$A \rightarrow B \rightarrow C$

For example:

T1: $A \rightarrow B$

T2: $B \rightarrow C$

T3: $A \rightarrow C$

This eliminates circular wait.

Conclusion

Deadlock occurs because transactions acquire resources in different orders. A **fixed global resource-locking order** is an effective solution.

Question 3 — Mobile OS App Switching Lag

1. Why does CPU remain idle?

Although 20 background apps are active, many processes may be **waiting for I/O**, such as:

- Disk access
- Network operations
- User input
- File operations

When all ready processes are waiting for I/O, the CPU has no process to execute and becomes idle.

Therefore, **many active processes do not necessarily mean that the CPU is always busy.**

2. Is quantum = 20 ms helping or hurting?

A **20 ms time quantum** with 20 background applications can cause frequent context switching.

When the quantum expires, the OS switches to another process. Too many switches cause:

- Context-switch overhead
- Cache disruption
- Increased scheduling overhead
- Reduced useful CPU work

Thus, in this scenario, the 20 ms quantum can **hurt performance** if it is too short for the workload.

3. Recommended solution

Use **Priority-based Round Robin** or increase the time quantum moderately, for example:

30–50 ms for background tasks.

Foreground applications such as WhatsApp and the music player can receive higher priority than background applications.

4. How does it improve performance?

Higher priority for active/foreground applications allows them to receive CPU time sooner.

A slightly larger quantum reduces excessive context switching.

Therefore:

Fewer context switches



Less scheduling overhead



More useful CPU execution



Better throughput



Smoother app switching

Conclusion

A **priority-based scheduling approach with a moderate quantum** is suitable because it gives responsive applications faster CPU access while reducing unnecessary context switching.

Question 4 — Deadlock Risk in an Operating System Update

1. Can deadlock occur?

Yes. The given resource allocation can lead to deadlock.

2. Circular wait condition

Given:

M1 holds R1 → waits for R2

M2 holds R2 → waits for R3

M3 holds R3 → waits for R1

The circular dependency is:

M1 → R2 → M2 → R3 → M3 → R1 → M1

Therefore, a **circular wait** exists.

The three modules are waiting for resources held by one another.

3. Banker's Algorithm-style allocation

Before granting a resource request:

1. Check whether the requested resource is available.
2. Temporarily allocate it.
3. Calculate the remaining resource needs.
4. Check whether all modules can eventually finish.
5. If a **safe sequence** exists, grant the request.
6. If no safe sequence exists, delay the request.

Example safe sequence:
 $M1 \rightarrow M2 \rightarrow M3$

The OS should grant resources only if the resulting state remains safe.

4. Prioritizing modules

The OS can give higher priority to critical update operations and schedule independent tasks first.

For example:

Critical update task



Higher CPU priority



Faster completion



CPU becomes available for other tasks

This improves CPU utilization while preventing unnecessary waiting.

Conclusion

Banker's Algorithm-style safe-state checking and proper module prioritization can reduce the risk of deadlock during system updates.

Question 5 — Cloud Server CPU Utilization Analysis

Given:

- I/O wait probability, $p = 80\% = 0.8$
- Number of VMs, $n = 7$

Formula:

$$U = 1 - p^n$$

1. Calculate CPU utilization

$$U = 1 - (0.8)^7$$
$$(0.8)^7 = 0.2097152$$

Therefore,

$$U = 1 - 0.2097152$$
$$U = 0.7902848$$

Percentage:

$$U \approx 79.03\%$$

2. Interpretation

The estimated CPU utilization is approximately **79.03%**.

This means the CPU is being used reasonably well, but there is still approximately:

$$100 - 79.03 = 20.97\%$$

of idle capacity.

Therefore, the server is **not fully utilized**, but it is reasonably loaded.

3. Effect of changing number of VMs

Using more VMs can increase CPU utilization because while some VMs wait for I/O, other VMs can use the CPU.

For example:

More VMs



More ready processes



Less CPU idle time



Higher throughput

However, too many VMs can cause CPU contention and reduce performance.

Reducing VMs may decrease CPU utilization and throughput if the server becomes underloaded.

4. Two OS-level strategies

1. CPU scheduling:

Use an efficient scheduling algorithm to give CPU time to ready VMs while others are waiting for I/O.

2. Load balancing:

Distribute workloads across available CPU cores/servers to prevent one CPU from becoming overloaded while others remain idle.

Conclusion

The calculated CPU utilization is **approximately 79.03%**. Increasing VMs moderately can improve utilization, but excessive VM consolidation may cause contention.

Question 6 — Hospital Emergency System Process Scheduling

1. Scheduling issue

The issue is **starvation**.

P1 has the highest priority and emergency requests continue to arrive. Therefore, low-priority **P3 (pharmacy billing)** may continuously wait.

2. Suitable CPU scheduling algorithm

Use **Priority Scheduling with Aging**.

Priority Scheduling ensures emergency processes receive quick CPU access, while aging prevents low-priority processes from waiting indefinitely.

3. Preventing starvation

Use **aging**.

Aging gradually increases the priority of a process that has been waiting for a long time.

For example:

P3 waiting



Priority gradually increases



P3 eventually gets CPU



Pharmacy billing completes

4. Response time and fairness

Emergency registration receives high priority, so its response time remains low.
At the same time, aging ensures P3 eventually receives CPU time.

Therefore:

- Emergency patients get fast service.
- Pharmacy billing is not ignored.
- CPU time is distributed more fairly.
- Starvation is reduced.

Conclusion

Priority Scheduling with Aging is suitable because it provides fast response for emergency operations while maintaining fairness for lower-priority tasks.

Question 7 — E-Commerce Warehouse Synchronization

1. Synchronization problem

The system has a **race condition**.

Multiple threads simultaneously access and modify the shared inventory database.

This can cause:

- Incorrect stock counts
- Duplicate order confirmations
- Inconsistent data

2. Why is a critical section required?

The inventory update operation is a **critical section** because it accesses shared data.

Only one thread should modify the same inventory record at a time.

For example:

Read stock

↓

Update stock

↓

Confirm order

These operations must be protected from simultaneous access.

3. Recommended solution — Mutex/Semaphore

A **binary semaphore (mutex)** can be used.

Example:

wait(mutex)

↓

Update inventory

↓

Confirm order

↓

signal(mutex)

Only one worker thread can enter the critical section at a time.

4. Maintaining consistency

When one thread updates the stock, other threads must wait until the operation is completed.

This prevents two workers from modifying the same inventory value simultaneously.

Therefore:

Mutual exclusion

↓
No race condition
↓
Correct stock count
↓
No duplicate confirmations
↓
Consistent database

Conclusion

Using a **binary semaphore/mutex** protects the critical section and maintains inventory consistency.

Question 8 — Video Streaming Service Thread Management

1. Thread-management problem

The problem is the **one-thread-per-client model**.

With **60,000 users**, creating 60,000 threads causes:

- High CPU usage
- High memory consumption
- Large context-switching overhead
- Poor scalability
- Video buffering

2. Suitable multithreading model

Use a **thread pool model**.

Instead of creating one thread for every client, maintain a limited number of worker threads.

60,000 Clients

↓
Task Queue
↓
Thread Pool

↓
Limited Worker Threads
Threads are reused for multiple client requests.

3. Two optimizations

Optimization 1: Thread Pool

Use a fixed or dynamically sized pool of worker threads instead of creating one thread per connection.

Optimization 2: Asynchronous / Non-blocking I/O

Use asynchronous I/O so threads do not remain blocked while waiting for network operations.

4. Effect on throughput

Thread pooling reduces:

- Thread creation overhead
- Memory usage
- Context switching

Asynchronous I/O allows a smaller number of threads to handle many connections efficiently.
Therefore:

Lower overhead



More available CPU & memory



More simultaneous clients



Higher throughput



Less buffering

Conclusion

A **thread pool with asynchronous I/O** provides better scalability and performance for a large number of simultaneous streaming users.

Question 9 — Smart Traffic Control Deadlock

1. Does deadlock exist?

Yes, a deadlock exists.

Given:

T1 holds R1 → requests R2

T2 holds R2 → requests R3

T3 holds R3 → requests R4

T4 holds R4 → requests R1

Circular wait:

T1 → R2 → T2 → R3 → T3 → R4 → T4 → R1 → T1

Therefore, none of the transactions can proceed.

2. Four necessary conditions

1. Mutual Exclusion

Each junction resource can be controlled by only one process at a time.

2. Hold and Wait

Each transaction holds one junction while waiting for another.

3. No Preemption

A held junction cannot simply be taken away from a transaction.

4. Circular Wait

There is a circular chain:

T1 → T2 → T3 → T4 → T1

All four conditions exist, so deadlock occurs.

3. Recovery technique

Use **process/resource termination**.

Terminate one transaction, for example T4, and release R4.

Then:

T3 can obtain R4



T3 completes



R3 is released



T2 completes

↓

T1 completes

The deadlock is broken.

4. Redesign to prevent deadlock

Use a **fixed resource ordering**.

For example:

$R1 \rightarrow R2 \rightarrow R3 \rightarrow R4$

Every transaction must request resources in this order.

This prevents circular wait and therefore prevents deadlock.

Conclusion

The four transactions form a circular wait. **Resource ordering** is a simple method to prevent similar deadlocks.

Question 10 — University Examination Server Resource Allocation

1. Applying Banker's Algorithm

Banker's Algorithm can be used to ensure that allocating CPU, memory, and database connections does not put the examination server into an unsafe state.

For every student session:

Request resources

↓

Check availability

↓

Pretend to allocate

↓

Check safe state

↓

Grant / Delay request

The request is granted only if the system remains in a **safe state**.

2. Information required

The system needs:

- **Available:** Currently available resources.
- **Maximum:** Maximum resources each process/session may need.
- **Allocation:** Resources currently allocated to each session.
- **Need:** Remaining resources required.

Formula:

$$Need = Maximum - Allocation$$

3. Determining a safe state

The algorithm temporarily assumes that a resource request is granted.

Then it checks whether there is a **safe sequence** in which all student sessions can obtain their remaining resources and finish.

If such a sequence exists:

Safe State

If no safe sequence exists:

Unsafe State

The request is delayed in the unsafe case.

4. Limitation

The major limitation is that Banker's Algorithm requires knowledge of the **maximum resource requirements** of processes in advance.

In a large cloud-based examination system, thousands of sessions may have changing and unpredictable resource requirements.

This can make the algorithm computationally expensive and difficult to manage at large scale.

Conclusion

Banker's Algorithm helps avoid deadlocks by granting resources only when the system remains in a safe state. However, its requirement for advance maximum-demand information makes it less practical for very large dynamic cloud systems.

